

GROUP EXERCISE — SOLUTIONS

ECE422 Computer Communications & Networking · Shannon · Checksum · CRC · Parity · Coding

Fields in Q2 are hexadecimal: a2 : 06 : 23 → 10100010 : 00000110 : 00100011

1. A radio link at 40 KHz can transmit a signal at 100 mW and the channel has 10 mW of noise.

What is the capacity of the channel?

$$S/N = 100 \text{ mW} / 10 \text{ mW} = \mathbf{10} \quad (= 10 \cdot \log_{10} 10 = 10 \text{ dB})$$

$$C = B \cdot \log_2(1 + S/N) = 40,000 \cdot \log_2(11)$$

$$\log_2(11) = \log_{10}(11) / \log_{10}(2) = 1.0414 / 0.3010 \approx 3.46$$

$$C \approx 40,000 \times 3.46 \approx \mathbf{138.4 \text{ kbps}}$$

2. Show the 8-bit checksum that will be transmitted to protect the fields a2:06:23.
-

<pre>a2 = 1010 0010 (162) + 06 = 0000 0110 (6) + 23 = 0010 0011 (35) ----- cb = 1100 1011 (203) no carry out of 8 bits → nothing to wrap Complement: 0011 0100 = 0x34 ← CHECKSUM Receiver: cb + 34 = 1111 1111 → complement = 0000 0000 → OK</pre>

Checksum = 0011 0100 (0x34). Transmitted: a2 : 06 : 23 : 34

3. Show 3 possible field values that would generate the same checksum as in 2).
-

The checksum depends only on the ones-complement **sum** (= cb). Any fields adding to cb give checksum 34:

#	Fields	Why
1	23 : 06 : a2	same values, different order (addition is commutative)
2	a3 : 05 : 23	+1 in one field, -1 in another
3	cb : 00 : 00	cb + 0 + 0 = cb

Lesson: a checksum cannot detect reordered fields or errors that cancel out.

4. What is the CRC field content in Q2 if $C(x) = 1101$?

$C(x) = 1101 = x^3 + x^2 + 1 \rightarrow$ degree $r = 3 \rightarrow$ append **000**.

Dividend: 10100010 00000110 00100011 **000**. Window method: 4-bit window; if it starts with 1 $\rightarrow$ XOR with 1101; drop the first bit; bring down the next bit.

```
1010 ⊕ 1101 = 0111 → 111, bring 0 → 1110
1110 ⊕ 1101 = 0011 → 011, bring 0 → 0110
0110 starts with 0 → no XOR → 110, bring 1 → 1101
1101 ⊕ 1101 = 0000 → 000, bring 0 → 0000
0000 starts with 0 → no XOR → 000, bring 0 → 0000
0000 starts with 0 → no XOR → 000, bring 0 → 0000
0000 starts with 0 → no XOR → 000, bring 0 → 0000
0000 starts with 0 → no XOR → 000, bring 0 → 0000
0000 starts with 0 → no XOR → 000, bring 0 → 0000
0000 starts with 0 → no XOR → 000, bring 1 → 0001
0001 starts with 0 → no XOR → 001, bring 1 → 0011
0011 starts with 0 → no XOR → 011, bring 0 → 0110
0110 starts with 0 → no XOR → 110, bring 0 → 1100
1100 ⊕ 1101 = 0001 → 001, bring 0 → 0010
0010 starts with 0 → no XOR → 010, bring 1 → 0101
0101 starts with 0 → no XOR → 101, bring 0 → 1010
1010 ⊕ 1101 = 0111 → 111, bring 0 → 1110
1110 ⊕ 1101 = 0011 → 011, bring 0 → 0110
0110 starts with 0 → no XOR → 110, bring 1 → 1101
1101 ⊕ 1101 = 0000 → 000, bring 1 → 0001
0001 starts with 0 → no XOR → 001, bring 0 → 0010
0010 starts with 0 → no XOR → 010, bring 0 → 0100
0100 starts with 0 → no XOR → 100, bring 0 → 1000
1000 ⊕ 1101 = 0101 → remainder = 101
```

CRC field = 101 $\rightarrow$ transmitted: 101000100000011000100011 **101** (dividing it by 1101 leaves 000 ✓)

5. If we use parity check for a 5-bit word, how many 2-bit errors may be undetectable?

A single parity bit misses **every** even number of bit errors $\rightarrow$ all 2-bit errors are undetectable. Count = number of ways to pick 2 bits:

- 5-bit word including the parity bit (like 1001P): $C(5,2) = 5 \cdot 4 / 2 = 10$
- 5 data bits + 1 parity bit (6 bits): $C(6,2) = 6 \cdot 5 / 2 = 15$ (*state your assumption*)

6. If we use 1 bit for error detection of 5-bit words, which of 1-bit parity (assuming up to 2-bit errors) or CRC would have fewer non-detectable errors?

A 1-bit CRC needs a generator of degree 1. The only useful one is $C(x) = x + 1$ (**11**) ($C(x) = x$ has one term and misses errors).

Dividing by $x + 1$ gives exactly the **even parity** bit $\rightarrow$ the two schemes are identical.

Errors (6 bits: 5 data + 1 check)	1-bit parity	CRC, $C(x)=x+1$
1-bit: $C(6,1) = 6$	all detected	all detected
2-bit: $C(6,2) = 15$	15 undetected	15 undetected

Answer: neither — same number of non-detectable errors (15). CRC only does better with more check bits.

7. How many 2-bit errors can be undetected by checksum in 5-bit words?

Assume one 5-bit data word + a 5-bit checksum (10 bits). The checksum is the complement of the word, so in every bit position the two bits are **opposite** (one 0, one 1):

data	:	d4	d3	d2	d1	d0			
checksum	:	~d4	~d3	~d2	~d1	~d0		data + checksum =	11111

Flipping the **same position** in both words → one goes 0 → 1, the other 1 → 0 → the sum is unchanged → **undetected**.

One such error per bit position → **5 undetectable 2-bit errors** (out of $C(10,2) = 45$).

Flips at different positions change the sum ($\pm 2^a \pm 2^b \neq 0 \pmod{31}$), so they are detected.

General rule: a 2-bit error escapes a checksum when two words flip the same bit position in opposite directions.

8. What is the CRC field content of the string in Q2 if $C(x) = 1000101$?

$C(x) = 1000101 = x^6 + x^2 + 1 \rightarrow$ degree $r = 6 \rightarrow$ append **000000**. 7-bit window:

1010001	⊕	1000101	=	0010100	→	010100, bring 0	→	0101000		
0101000						starts with 0	→	no XOR → 101000, bring 0	→	1010000
1010000	⊕	1000101	=	0010101	→	010101, bring 0	→	0101010		
0101010						starts with 0	→	no XOR → 101010, bring 0	→	1010100
1010100	⊕	1000101	=	0010001	→	010001, bring 0	→	0100010		
0100010						starts with 0	→	no XOR → 100010, bring 0	→	1000100
1000100	⊕	1000101	=	0000001	→	000001, bring 1	→	0000011		
0000011						starts with 0	→	no XOR → 000011, bring 1	→	0000111
0000111						starts with 0	→	no XOR → 000111, bring 0	→	0001110
0001110						starts with 0	→	no XOR → 001110, bring 0	→	0011100
0011100						starts with 0	→	no XOR → 011100, bring 0	→	0111000
0111000						starts with 0	→	no XOR → 111000, bring 1	→	1110001
1110001	⊕	1000101	=	0110100	→	110100, bring 0	→	1101000		
1101000	⊕	1000101	=	0101101	→	101101, bring 0	→	1011010		
1011010	⊕	1000101	=	0011111	→	011111, bring 0	→	0111110		
0111110						starts with 0	→	no XOR → 111110, bring 1	→	1111101
1111101	⊕	1000101	=	0111000	→	111000, bring 1	→	1110001		
1110001	⊕	1000101	=	0110100	→	110100, bring 0	→	1101000		
1101000	⊕	1000101	=	0101101	→	101101, bring 0	→	1011010		
1011010	⊕	1000101	=	0011111	→	011111, bring 0	→	0111110		
0111110						starts with 0	→	no XOR → 111110, bring 0	→	1111100
1111100	⊕	1000101	=	0111001	→	111001, bring 0	→	1110010		
1110010	⊕	1000101	=	0110111	→	110111, bring 0	→	1101110		
1101110	⊕	1000101	=	0101011	→	101011, bring 0	→	1010110		

CRC field = 101011 → transmitted: 101000100000011000100011 **101011** (remainder check = 000000

✓)

9. (Optional) Design a coding scheme to transmit 4-bit words (16 combinations) where a string of words cannot contain more than 4 consecutive 1s or 0s.

Idea (like 4B/5B): map each 4-bit word to a **5-bit code** that does **not start** and does **not end** with three equal bits.

Then at the boundary between two codes at most $2 + 2 = 4$ equal bits can line up, and inside a code the longest run is 3.

18 of the 32 five-bit codes satisfy the rule ($32 - 8 - 8 + 2$); choose 16:

Data	Code	Data	Code	Data	Code	Data	Code
0000	00100	0100	01010	1000	01110	1100	10100
0001	00101	0101	01011	1001	10001	1101	10101
0010	00110	0110	01100	1010	10010	1110	10110
0011	01001	0111	01101	1011	10011	1111	11001

Checked: every pair of codes back-to-back has a longest run of 4. Efficiency = $4/5 = 80\%$. The 2 unused codes (11010, 11011) can serve as control codes.

Key rules: dB $\rightarrow$ S/N: divide by 10, take 10 to that power · checksum: add, wrap carry, complement · CRC: append r zeros ($r = \text{degree} = \text{bits of } C(x) - 1$), XOR-divide, remainder = CRC · parity misses all even-bit errors, count with $C(n,k)$.